Deploy NodeConnect (Custom Build) on DigitalOcean Kubernetes

This guide walks through deploying **NodeConnect** (a custom Activepieces build with the `ada-bmp` piece) on DigitalOcean Kubernetes (DOKS). It is based on the actual deployment to **nodesconnect.in** and includes all gotchas and fixes encountered.

Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                        DigitalOcean Cloud                        │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                 DOKS Kubernetes Cluster (nyc1)           │   │
│  │                                                          │   │
│  │   [DO Load Balancer]                                     │   │
│  │         │                                                │   │
│  │         ▼                                                │   │
│  │   [Ingress-NGINX + cert-manager (HTTPS)]                 │   │
│  │         │                                                │   │
│  │         ▼                                                │   │
│  │   [NodeConnect StatefulSet — 2 replicas]                 │   │
│  │     (ghcr.io/YOUR_USERNAME/nodeconnect-app:latest)       │   │
│  │     Each pod runs WORKER_AND_APP mode via PM2:           │   │
│  │       ├─ activepieces-app    (API server on port 3001)   │   │
│  │       └─ activepieces-worker (job executor via Socket.IO) │   │
│  │             │                    │                       │   │
│  │             ▼                    ▼                       │   │
│  │   [In-cluster Redis]        [DO Managed Postgres]        │   │
│  │   (Bitnami Helm)             (external, SSL)             │   │
│  │   ├─ Job queues (BullMQ)    ├─ Flows, users, connections │   │
│  │   ├─ Worker registration    └─ Flow run history          │   │
│  │   └─ Cache                                               │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  [GHCR] ◄── Docker image pushed from local Mac (linux/amd64)     │
└──────────────────────────────────────────────────────────────────┘

Monitoring:
  ├─ BullMQ Dashboard: https://YOUR_DOMAIN/api/ui
  ├─ Workers page:     https://YOUR_DOMAIN/platform/infrastructure/workers
  └─ DBeaver:          Connect to DO Managed Postgres (port 25060, SSL)
```

System Requirements

Cloud Infrastructure (DigitalOcean)

Component	Spec	Notes
K8s Nodes (x2)	2 vCPU, 4 GB RAM each	`s-2vcpu-4gb` — handles API + Worker per pod
Managed PostgreSQL	1 vCPU, 1 GB RAM, 10 GB disk	`db-s-1vcpu-1gb` — stores flows, users, run history
In-cluster Redis	2 GB persistent volume	Standalone Bitnami chart on existing nodes
Load Balancer	DO managed	Provisioned automatically by Ingress-NGINX

Component	Spec	Notes
Block Storage	10 GB per pod	`do-block-storage` for app cache

Local Build Machine

Requirement	Minimum	Notes
Docker	Docker Desktop 4.x+	BuildKit enabled (default)
Disk space	~15 GB free	Docker image build uses ~5 GB; caches ~10 GB
RAM	8 GB	Docker build can peak at ~4 GB
CPU	4 cores recommended	Faster builds; cross-compilation (`linux/amd64`) is slower on Apple Silicon
Node.js	v20+	For local Nx commands if needed
OS	macOS, Linux, or WSL2	Apple Silicon Macs require `--platform linux/amd64`

Application Resource Usage (per pod)

Resource	Request (guaranteed)	Limit (max)
CPU	250m (0.25 cores)	1000m (1 core)
Memory	512 Mi	2 Gi

With 2 replicas, the total cluster footprint is:

- **CPU:** 0.5–2.0 cores for NodeConnect + overhead for Redis, ingress, system pods
- **Memory:** 1–4 GB for NodeConnect + ~0.5 GB for Redis + system overhead
- **Storage:** 20 GB block storage (10 GB x 2 pods) + 2 GB Redis persistence

Software Stack

Layer	Technology	Version
Runtime	Node.js	20.x
Process Manager	PM2	latest
Framework	Fastify	5.x
ORM	TypeORM	latest
Job Queue	BullMQ (Redis-backed)	latest
Frontend	React 18 + Vite	18.x
Database	PostgreSQL	16
Cache/Queue	Redis	7.x
Container	Docker (BuildKit)	latest
Orchestration	Kubernetes (DOKS)	latest
Package Manager	Helm	3.x
SSL	cert-manager + Let's Encrypt	1.13+
Ingress	NGINX Ingress Controller	latest
Registry	GitHub Container Registry (GHCR)	—

Prerequisites Summary

Tool	Purpose	Install (Mac)
`doctl`	DigitalOcean CLI	`brew install doctl`
`kubectl`	Kubernetes CLI	`brew install kubectl`
`helm`	Kubernetes package manager	`brew install helm`
`docker`	Build container images	Docker Desktop
`openssl`	Generate security keys	`brew install openssl`

Step 1: Install Required Tools

Mac (Homebrew):

```
brew install doctl kubectl helm openssl
```

Verify:

```
doctl version
kubectl version --client
helm version
docker version
```

Step 2: Create DigitalOcean Account and Connect

1. Create account at digitalocean.com and add a payment method.
2. Go to **API** in the left sidebar → **Generate New Token** → Name: `nodeconnect-deploy` → Full Access → Copy the token.
3. Connect `doctl`:

```
doctl auth init
# Paste your token when prompted
```

Verify:

```
doctl account get
```

Step 3: Build and Push Your Custom Docker Image

3a: Dockerfile Requirements

Your `Dockerfile` must include the BMP piece build step with a BuildKit cache mount. The key line is:

```
RUN --mount=type=cache,target=/root/.bun/install/cache npx nx build pieces-ada-bmp --skip-nx-cache
```

> **Why the cache mount?** The `pieces-ada-bmp` prebuild target runs `bun install`, which needs cached package manifests. Without the cache mount, `bun install` fails with `ConnectionRefused` during the Docker build because there is no network access to the bun registry for cached packages.

Also ensure the Dockerfile mirrors the nx output into the expected location:

```
RUN if [ -d dist/packages/pieces/custom/ada-bmp ]; then \
      mkdir -p packages/pieces/custom/ada-bmp/dist && \
      cp -a dist/packages/pieces/custom/ada-bmp/. packages/pieces/custom/ada-bmp/dist/; \
    fi
```

3b: Create a GitHub Personal Access Token (PAT)

1. Go to github.com/settings/tokens
2. **Generate new token (classic)** → Name: `nodeconnect-registry`
3. Scopes: `write:packages` , `read:packages` , `delete:packages`
4. Copy the token immediately.

3c: Login to GitHub Container Registry

```
# IMPORTANT: GitHub username MUST be lowercase
export GITHUB_USERNAME="your-lowercase-github-username"

echo "YOUR_GITHUB_PAT" | docker login ghcr.io -u $GITHUB_USERNAME --password-stdin
```

3d: Build the Docker Image

> **CRITICAL for Apple Silicon (M1/M2/M3 Macs):** You MUST build for `linux/amd64` because DOKS nodes run AMD64. Without `--platform linux/amd64` , the pods will crash with `exec format error` .

```
cd /path/to/your/activepieces-bmp-custom

# Build the Docker image for AMD64 (required even on Apple Silicon Macs)
# The Dockerfile already builds the ada-bmp piece internally — no local pre-build needed.
docker build --platform linux/amd64 -t nodeconnect-app:latest -f Dockerfile .
```

This takes 15–25 minutes on first build.

3e: Tag and Push

```
docker tag nodeconnect-app:latest ghcr.io/$GITHUB_USERNAME/nodeconnect-app:latest
docker push ghcr.io/$GITHUB_USERNAME/nodeconnect-app:latest
```

3f: Verify on GitHub

Go to `https://github.com/users/YOUR_USERNAME/packages/container/package/nodeconnect-app` and confirm the image is listed. Keep it **Private** (recommended).

Step 4: Create Your Kubernetes Cluster

```
doctl kubernetes cluster create nodeconnect-cluster \
  --region nyc1 \
  --node-pool "name=worker-pool;size=s-2vcpu-4gb;count=2" \
  --version latest
```

This takes 5–10 minutes. Once done, connect `kubectl` :

```
doctl kubernetes cluster kubeconfig save nodeconnect-cluster
```

Verify:

```
kubectl get nodes
# Should show 2 nodes with STATUS "Ready"
```

Step 5: Create Databases

5a: Create Managed PostgreSQL

```
doctl databases create nodeconnect-postgres \
  --engine pg \
  --region nyc1 \
  --size db-s-1vcpu-1gb \
  --num-nodes 1 \
  --version 16
```

Wait 3–5 minutes, then get connection details:

```
doctl databases connection nodeconnect-postgres
```

Save: `Host` , `Port` , `User` , `Password` , `Database` .

> **SSL Note:** DigitalOcean managed Postgres requires SSL. We handle this with `AP_POSTGRES_USE_SSL=true` plus `NODE_TLS_REJECT_UNAUTHORIZED=0` (see Step 7).
> For production, download the CA certificate from the DO dashboard and set `AP_POSTGRES_SSL_CA` .

5b: Deploy In-Cluster Redis (Bitnami Helm Chart)

> **Why in-cluster?** DigitalOcean managed Redis is not available in all regions (e.g., `nyc1`).
> An in-cluster Redis via Bitnami is simple and avoids region availability issues.

```
# Create the namespace first (shared with the app)
kubectl create namespace nodeconnect

# Add Bitnami repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Install Redis (use SINGLE quotes around password to avoid bash expansion of special chars)
helm install nodeconnect-redis bitnami/redis \
  --namespace nodeconnect \
  --set auth.password='YourSecureRedisPassword!' \
  --set architecture=standalone \
  --set master.persistence.size=2Gi
```

Redis connection details for secrets:

Setting	Value
`AP_REDIS_HOST`	`nodeconnect-redis-master.nodeconnect.svc.cluster.local`
`AP_REDIS_PORT`	`6379`
`AP_REDIS_PASSWORD`	(the password you set above)
`AP_REDIS_USE_SSL`	`false`
`AP_REDIS_USER`	`default`

Setting	Value
AP_REDIS_TYPE	DEFAULT

Step 6: Set Up Ingress Controller and HTTPS

6a: Install Ingress-NGINX

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.service.annotations."service\.beta\.kubernetes\.io/do-loadbalancer-name"="nodeconnect-lb" \
  --set controller.service.annotations."service\.beta\.kubernetes\.io/do-loadbalancer-protocol"="http"
```

6b: Wait for Load Balancer IP

```
kubectl get svc -n ingress-nginx ingress-nginx-controller --watch
# Wait until EXTERNAL-IP changes from <pending> to an actual IP. Press Ctrl+C to stop.
```

Save this IP — you need it for DNS (Step 9).

6c: Install cert-manager (for HTTPS/Let's Encrypt)

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.13.0/cert-manager.yaml

# Wait for it to be ready
kubectl wait --for=condition=Available deployment --all -n cert-manager --timeout=300s
```

6d: Create the ClusterIssuer

```
cat << 'EOF' | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: your-email@example.com
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
    - http01:
        ingress:
          class: nginx
EOF
```

Step 7: Create Kubernetes Secrets

7a: Generate Security Keys

```
echo "AP_ENCRYPTION_KEY: $(openssl rand -hex 16)"
echo "AP_JWT_SECRET: $(openssl rand -hex 32)"
```

Save both values. (`AP_WORKER_TOKEN` is auto-generated at container startup from the JWT secret.)

7b: Set Environment Variables

```
# PostgreSQL (from Step 5a)
export DB_HOST="your-db-xxxxx.db.ondigitalocean.com"
export DB_PORT="25060"
export DB_USER="doadmin"
export DB_PASSWORD="AVNS_xxxxxxxxxxxxxxx"
export DB_NAME="defaultdb"

# Redis (from Step 5b)
export REDIS_HOST="nodeconnect-redis-master.nodeconnect.svc.cluster.local"
export REDIS_PORT="6379"
export REDIS_PASSWORD="YourSecureRedisPassword!"

# Security keys (from 7a)
export ENCRYPTION_KEY="paste-here"
export JWT_SECRET="paste-here"

# Domain
export MY_DOMAIN="nodesconnect.in"

# GitHub (lowercase)
export GITHUB_USERNAME="your-lowercase-github-username"
```

7c: Create the Image Pull Secret

```
kubectl create secret docker-registry ghcr-secret \
  --namespace nodeconnect \
  --docker-server=ghcr.io \
  --docker-username=$GITHUB_USERNAME \
  --docker-password=YOUR_GITHUB_PAT \
  --docker-email=your-email@example.com
```

7d: Create All Application Secrets

```
# Config secrets (includes BMP and custom piece config)
kubectl create secret generic nodeconnect-config-secrets \
  --namespace nodeconnect \
  --from-literal=AP_EDITION=ce \
  --from-literal=AP_EXECUTION_MODE=UNSANDBOXED \
  --from-literal=AP_ENVIRONMENT=production \
  --from-literal=AP_PORT=3001 \
  --from-literal=AP_FRONTEND_URL=https://$MY_DOMAIN \
  --from-literal=AP_ENABLE_FLOW_ON_PUBLISH=true \
  --from-literal=AP_CLIENT_REAL_IP_HEADER=X-Forwarded-For \
  --from-literal=AP_DB_TYPE=POSTGRES \
  --from-literal=AP_ENGINE_EXECUTABLE_PATH="dist/packages/engine/main.js" \
  --from-literal=AP_PIECES_SYNC_MODE=OFFICIAL_AUTO \
  --from-literal=AP_PIECES_SOURCE=FILE \
  --from-literal=AP_DEV_PIECES=ada-bmp \
  --from-literal=AP_BMP_ENABLED=true \
  --from-literal=AP_BMP_ORGANIZATIONS=true \
  --from-literal=AP_BMP_SUPER_ADMIN=true \
  --from-literal=AP_BMP_ACCOUNT_SWITCHING=true \
  --from-literal=AP_MULTI_TENANT_MODE=true \
  --from-literal=ADA_BMP_API_URL=https://bmpapi.bmp.ada-asia.my \
  --from-literal=ADA_BMP_TIMEOUT=30000 \
  --from-literal=AP_SANDBOX_PROPAGATED_ENV_VARS=ADA_BMP_API_URL \
  --from-literal=AP_WEBHOOK_TIMEOUT_SECONDS=30 \
  --from-literal=AP_TRIGGER_DEFAULT_POLL_INTERVAL=1

# Auth secrets
# IMPORTANT: Leave AP_WORKER_TOKEN empty — the docker-entrypoint.sh auto-generates
# a valid JWT from AP_JWT_SECRET at startup. Setting it to a random string breaks
# worker registration because it expects a properly signed JWT.
kubectl create secret generic nodeconnect-auth-secrets \
  --namespace nodeconnect \
  --from-literal=AP_ENCRYPTION_KEY=$ENCRYPTION_KEY \
  --from-literal=AP_JWT_SECRET=$JWT_SECRET \
  --from-literal=AP_API_KEY="" \
  --from-literal=AP_WORKER_TOKEN="" \
  --from-literal=AP_GOOGLE_CLIENT_ID="" \
  --from-literal=AP_GOOGLE_CLIENT_SECRET="" \
  --from-literal=AP_FIREBASE_HASH_PARAMETERS=""

# Database secrets
kubectl create secret generic nodeconnect-db-secrets \
  --namespace nodeconnect \
  --from-literal=AP_POSTGRES_DATABASE=$DB_NAME \
  --from-literal=AP_POSTGRES_HOST=$DB_HOST \
  --from-literal=AP_POSTGRES_PORT=$DB_PORT \
  --from-literal=AP_POSTGRES_PASSWORD=$DB_PASSWORD \
  --from-literal=AP_POSTGRES_USERNAME=$DB_USER \
  --from-literal=AP_POSTGRES_USE_SSL=true \
  --from-literal=AP_POSTGRES_POOL_SIZE="" \
  --from-literal=AP_POSTGRES_SSL_CA=""

# Redis secrets (in-cluster, no SSL)
kubectl create secret generic nodeconnect-redis-secrets \
  --namespace nodeconnect \
  --from-literal=AP_REDIS_TYPE=DEFAULT \
  --from-literal=AP_REDIS_HOST=$REDIS_HOST \
  --from-literal=AP_REDIS_PORT=$REDIS_PORT \
  --from-literal=AP_REDIS_PASSWORD=$REDIS_PASSWORD \
  --from-literal=AP_REDIS_USE_SSL=false \
  --from-literal=AP_REDIS_USER=default

# Queue secrets
```

```
kubectl create secret generic nodeconnect-queue-secrets \
  --namespace nodeconnect \
  --from-literal=AP_QUEUE_MODE=REDIS \
  --from-literal=AP_QUEUE_UI_ENABLED=false \
  --from-literal=AP_QUEUE_UI_USERNAME="" \
  --from-literal=AP_QUEUE_UI_PASSWORD=""

# Log secrets
kubectl create secret generic nodeconnect-log-secrets \
  --namespace nodeconnect \
  --from-literal=AP_LOG_LEVEL=info \
  --from-literal=AP_LOG_PRETTY=false \
  --from-literal=AP_SENTRY_DSN="" \
  --from-literal=AP_HYPERDX_TOKEN=""

# Telemetry secrets
# IMPORTANT: AP_OTEL_ENABLED must be explicitly "false", NOT an empty string
kubectl create secret generic nodeconnect-telemetry-secrets \
  --namespace nodeconnect \
  --from-literal=AP_TELEMETRY_ENABLED=false \
  --from-literal=AP_OTEL_ENABLED=false \
  --from-literal=AP_FEATUREBASE_API_KEY=""

# Remaining secrets (required by the Helm chart, can be empty)
for secret in nodeconnect-limits-secrets nodeconnect-cloudflare-secrets \
  nodeconnect-integrations-secrets nodeconnect-s3-secrets \
  nodeconnect-smtp-secrets nodeconnect-stripe-secrets nodeconnect-otel-secrets; do
  kubectl create secret generic $secret --namespace nodeconnect --from-literal=placeholder=""
done
```

> **Gotcha — AP_OTEL_ENABLED:** This variable expects exactly `"true"` or `"false"`. An empty string causes the app to crash with `EnvVarError`. Always set it explicitly to `false`.

> **Gotcha — AP_WORKER_TOKEN:** Do NOT set this to a random string. Leave it empty (`""`) so the `docker-entrypoint.sh` auto-generates a valid JWT from `AP_JWT_SECRET`. A non-JWT value prevents the worker from registering with the API.

> **Gotcha — AP_PLATFORM_ID_FOR_DEDICATED_WORKER:** Do NOT include this variable at all (not even as empty string). An empty string `""` causes the worker to register as `DEDICATED` type with no platform, making it invisible on the Workers page. When omitted, the worker registers as `SHARED` and appears correctly.

Verify:

```
kubectl get secrets -n nodeconnect
```

Step 8: Deploy with Helm

8a: Navigate to Your Helm Chart

Use your local repository that already contains the Activepieces Helm chart:

```
cd /path/to/your/activepieces-bmp-custom/deploy/activepieces-helm
```

8b: Create `values-digitalocean.yaml`

Create this file in the Helm chart directory. Replace placeholders with your actual values:

```yaml
replicaCount: 2

image:
  repository: ghcr.io/YOUR_GITHUB_USERNAME/nodeconnect-app
  pullPolicy: Always
  tag: "latest"

imagePullSecrets:
  - name: ghcr-secret

workloadType: statefulset

container:
  port: 3001

resources:
  limits:
    cpu: "1000m"
    memory: "2Gi"
  requests:
    cpu: "250m"
    memory: "512Mi"

persistence:
  enabled: true
  size: 10Gi
  storageClassName: do-block-storage
  mountPath: "/usr/src/app/cache"

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: "100m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "900"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "900"
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "60"
  hosts:
    - host: YOUR_DOMAIN
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: nodeconnect-tls
      hosts:
        - YOUR_DOMAIN

activepieces:
  frontendUrl: "https://YOUR_DOMAIN"
  edition: "ce"
  executionMode: "UNSANDBOXED"
  environment: "prod"
  telemetryEnabled: false

postgresql:
  enabled: false

redis:
  enabled: false
```

```yaml
autoscaling:
  enabled: false

# NODE_TLS_REJECT_UNAUTHORIZED=0 bypasses self-signed cert errors for DO managed Postgres.
# For production, download the CA cert from DO dashboard and use AP_POSTGRES_SSL_CA instead.
activepiecesConfig:
  AP_CONTAINER_TYPE: ""
  NODE_TLS_REJECT_UNAUTHORIZED: "0"

# IMPORTANT: Null out ALL default activepieces-*-secrets from values.yaml.
# Without this, Helm merges both sets and Kubernetes rejects the deployment
# with "duplicate entries for key" errors.
activepiecesEnvVariables:
  activepieces-config-secrets: null
  activepieces-auth-secrets: null
  activepieces-queue-secrets: null
  activepieces-log-secrets: null
  activepieces-telemetry-secrets: null
  activepieces-limits-secrets: null
  activepieces-cloudflare-secrets: null
  activepieces-integrations-secrets: null
  activepieces-db-secrets: null
  activepieces-redis-secrets: null
  activepieces-s3-secrets: null
  activepieces-smtp-secrets: null
  activepieces-stripe-secrets: null
  activepieces-otel-secrets: null

  nodeconnect-config-secrets:
    - AP_EDITION
    - AP_EXECUTION_MODE
    - AP_ENVIRONMENT
    - AP_PORT
    - AP_FRONTEND_URL
    - AP_ENABLE_FLOW_ON_PUBLISH
    - AP_CLIENT_REAL_IP_HEADER
    - AP_DB_TYPE
    - AP_ENGINE_EXECUTABLE_PATH
    - AP_PIECES_SYNC_MODE
    - AP_PIECES_SOURCE
    - AP_DEV_PIECES
    - AP_BMP_ENABLED
    - AP_BMP_ORGANIZATIONS
    - AP_BMP_SUPER_ADMIN
    - AP_BMP_ACCOUNT_SWITCHING
    - AP_MULTI_TENANT_MODE
    - ADA_BMP_API_URL
    - ADA_BMP_TIMEOUT
    - AP_SANDBOX_PROPAGATED_ENV_VARS
    - AP_WEBHOOK_TIMEOUT_SECONDS
    - AP_TRIGGER_DEFAULT_POLL_INTERVAL
  nodeconnect-auth-secrets:
    - AP_ENCRYPTION_KEY
    - AP_JWT_SECRET
    - AP_API_KEY
    - AP_WORKER_TOKEN
    - AP_GOOGLE_CLIENT_ID
    - AP_GOOGLE_CLIENT_SECRET
    - AP_FIREBASE_HASH_PARAMETERS
  nodeconnect-queue-secrets:
    - AP_QUEUE_MODE
    - AP_QUEUE_UI_ENABLED
    - AP_QUEUE_UI_USERNAME
    - AP_QUEUE_UI_PASSWORD
  nodeconnect-log-secrets:
```

```yaml
  - AP_LOG_LEVEL
  - AP_LOG_PRETTY
  - AP_SENTRY_DSN
  - AP_HYPERDX_TOKEN
nodeconnect-telemetry-secrets:
  - AP_TELEMETRY_ENABLED
  - AP_OTEL_ENABLED
  - AP_FEATUREBASE_API_KEY
nodeconnect-limits-secrets:
  - AP_MAX_CONCURRENT_JOBS_PER_PROJECT
  - AP_PROJECT_RATE_LIMITER_ENABLED
  - AP_API_RATE_LIMIT_AUTHN_ENABLED
  - AP_MAX_FILE_SIZE_MB
  - AP_MAX_FLOW_RUN_LOG_SIZE_MB
  - AP_EXECUTION_DATA_RETENTION_DAYS
nodeconnect-cloudflare-secrets:
  - AP_CLOUDFLARE_API_BASE
  - AP_CLOUDFLARE_ZONE_ID
  - AP_CLOUDFLARE_API_TOKEN
nodeconnect-integrations-secrets:
  - AP_APP_WEBHOOK_SECRETS
  - AP_SECRET_MANAGER_API_KEY
  - AP_OPENROUTER_PROVISION_KEY
  - AP_TEMPLATES_API_KEY
  - AP_TEMPLATE_MANAGER_API_KEY
nodeconnect-db-secrets:
  - AP_POSTGRES_DATABASE
  - AP_POSTGRES_HOST
  - AP_POSTGRES_PORT
  - AP_POSTGRES_PASSWORD
  - AP_POSTGRES_USERNAME
  - AP_POSTGRES_USE_SSL
  - AP_POSTGRES_POOL_SIZE
  - AP_POSTGRES_SSL_CA
nodeconnect-redis-secrets:
  - AP_REDIS_TYPE
  - AP_REDIS_HOST
  - AP_REDIS_PORT
  - AP_REDIS_PASSWORD
  - AP_REDIS_USE_SSL
  - AP_REDIS_USER
nodeconnect-s3-secrets:
  - AP_FILE_STORAGE_LOCATION
  - AP_S3_BUCKET
  - AP_S3_REGION
  - AP_S3_ENDPOINT
  - AP_S3_USE_SIGNED_URLS
  - AP_S3_ACCESS_KEY_ID
  - AP_S3_SECRET_ACCESS_KEY
nodeconnect-smtp-secrets:
  - AP_SMTP_HOST
  - AP_SMTP_PORT
  - AP_SMTP_USERNAME
  - AP_SMTP_PASSWORD
  - AP_SMTP_SENDER_EMAIL
  - AP_SMTP_SENDER_NAME
nodeconnect-stripe-secrets:
  - AP_STRIPE_SECRET_KEY
  - AP_STRIPE_WEBHOOK_SECRET
nodeconnect-otel-secrets:
  - OTEL_EXPORTER_OTLP_ENDPOINT
  - OTEL_EXPORTER_OTLP_HEADERS
```

8c: Update Helm Dependencies and Deploy

```
helm dependency update

helm install nodeconnect . \
  --namespace nodeconnect \
  --values values-digitalocean.yaml
```

Expected output:

```
NAME: nodeconnect
LAST DEPLOYED: ...
NAMESPACE: nodeconnect
STATUS: deployed
```

8d: Watch Pods Start

```
kubectl get pods -n nodeconnect --watch
```

Wait until all pods show `1/1 Running` . Press Ctrl+C to stop watching.

> **If pods show `CrashLoopBackOff` :**

```
> kubectl logs -n nodeconnect nodeconnect-activepieces-0
>
```

> Common causes:
>
> - `exec format error` → rebuild image with `--platform linux/amd64`
> - `EnvVarError: "AP_OTEL_ENABLED"` → recreate `nodeconnect-telemetry-secrets` with `AP_OTEL_ENABLED=false`
> - `self-signed certificate in certificate chain` → ensure `NODE_TLS_REJECT_UNAUTHORIZED: "0"` is in `activepiecesConfig`

Step 9: Configure Your Domain (DNS)

9a: Get Load Balancer IP

```
kubectl get svc -n ingress-nginx ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

9b: Configure DNS Records

At your DNS provider (e.g., Cloudflare), create:

Type	Name	Value	Proxy
A	@	LOAD_BALANCER_IP	Off
A	www	LOAD_BALANCER_IP	Off

> **Cloudflare users:** Turn proxy **off** (DNS only / grey cloud) so cert-manager can complete the HTTP-01 challenge. You can enable proxy after the certificate is issued.

9c: Wait for DNS Propagation

```
nslookup YOUR_DOMAIN
# Should resolve to your load balancer IP
```

DNS propagation typically takes 5–30 minutes.

Step 10: Verify Everything is Working

10a: Check Pods

```
kubectl get pods -n nodeconnect
# All pods should be 1/1 Running
```

10b: Check Live Logs

```
kubectl logs -n nodeconnect nodeconnect-activepieces-0 -f
```

Look for:

```
info -- [ServerApi#start] 🚀 App is listening at 0.0.0.0:3001
```

10c: Check the Ingress and Certificate

```
kubectl get ingress -n nodeconnect
kubectl get certificate -n nodeconnect
# READY should be True
```

10d: Verify Custom Piece is Loaded

```
# Check piece files exist in the container
kubectl exec -n nodeconnect nodeconnect-activepieces-0 -- \
  ls -la /usr/src/app/packages/pieces/custom/ada-bmp/dist/

# Check env vars are correct
kubectl exec -n nodeconnect nodeconnect-activepieces-0 -- \
  env | grep -E "AP_PIECES_SOURCE|AP_DEV_PIECES"
```

Expected:

```
AP_PIECES_SOURCE=FILE
AP_DEV_PIECES=ada-bmp
```

10e: Access the Application

Open `https://YOUR_DOMAIN` in your browser. Create your first admin account, then create a flow and verify the **ada-bmp** piece appears in the piece selector.

How to Update After Code Changes

Rebuild and Push

```
cd /path/to/your/activepieces-bmp-custom

# Build the Docker image for AMD64 (always required for DOKS)
# The Dockerfile already builds the ada-bmp piece internally.
docker build --platform linux/amd64 -t nodeconnect-app:latest -f Dockerfile .

# Tag and push
docker tag nodeconnect-app:latest ghcr.io/$GITHUB_USERNAME/nodeconnect-app:latest
docker push ghcr.io/$GITHUB_USERNAME/nodeconnect-app:latest
```

Deploy the Update

Option A — Restart pods (if using `latest` tag):

```
kubectl rollout restart statefulset nodeconnect-activepieces -n nodeconnect
kubectl rollout status statefulset nodeconnect-activepieces -n nodeconnect
```

Option B — Helm upgrade (if you changed values):

```
cd /path/to/your/activepieces-bmp-custom/deploy/activepieces-helm

helm upgrade nodeconnect . \
  --namespace nodeconnect \
  --values values-digitalocean.yaml
```

Update a Kubernetes Secret

To change a secret (e.g., `nodeconnect-config-secrets`):

```
# Delete the old one
kubectl delete secret nodeconnect-config-secrets -n nodeconnect

# Recreate with updated values
kubectl create secret generic nodeconnect-config-secrets \
  --namespace nodeconnect \
  --from-literal=KEY1=value1 \
  --from-literal=KEY2=value2 \
  ...

# Restart pods to pick up changes
kubectl delete pod nodeconnect-activepieces-0 nodeconnect-activepieces-1 -n nodeconnect
```

Troubleshooting Reference

Symptom	Cause	Fix
`exec format error` in pod logs	Image built for ARM64, DOKS needs AMD64	Rebuild with `--platform linux/amd64`
`EnvVarError: "AP_OTEL_ENABLED"`	Empty string instead of `false`	Recreate `nodeconnect-telemetry-secrets` wit `AP_OTEL_ENABLED=false`
`self-signed certificate in certificate chain`	DO managed Postgres uses self-signed SSL	Set `NODE_TLS_REJECT_UNAUTHORIZED: "0"` in `activepiecesConfig`

Symptom	Cause	Fix
`duplicate entries for key` on helm install	Default secrets from `values.yaml` merge with custom secrets	Null out all `activepieces-*-secrets` in your va
`cannot reuse a name that is still in use`	Previous failed Helm release still exists	`helm uninstall nodeconnect -n nodeconnect`
`CrashLoopBackOff`	Various — check logs	`kubectl logs -n nodeconnect nodeconnect-a`
Custom piece not visible	`AP_PIECES_SOURCE` / `AP_DEV_PIECES` not set	Set `AP_PIECES_SOURCE=FILE` and `AP_DEV_PIEC`
Workers page "No workers found"	`AP_WORKER_TOKEN` set to random string (not JWT)	Leave `AP_WORKER_TOKEN=""` so entrypoint auto- valid JWT
Workers page "No workers found"	`AP_PLATFORM_ID_FOR_DEDICATED_WORKER=""`	Remove this variable entirely — empty string reg DEDICATED
Redis password with `!` causes `dquote>`	Bash history expansion	Use single quotes: `--set auth.password='Pass`
`ConnectionRefused` during Docker build (bun install)	Missing BuildKit cache mount	Add `--mount=type=cache,target=/root/.bun/inst` the RUN line

Useful Commands

```
# View all resources in the namespace
kubectl get all -n nodeconnect

# Live logs for a pod
kubectl logs -n nodeconnect nodeconnect-activepieces-0 -f

# Exec into a pod for debugging
kubectl exec -it -n nodeconnect nodeconnect-activepieces-0 -- /bin/sh

# Check all env vars in a pod
kubectl exec -n nodeconnect nodeconnect-activepieces-0 -- env | sort

# Restart all app pods
kubectl rollout restart statefulset nodeconnect-activepieces -n nodeconnect

# Delete and reinstall via Helm
helm uninstall nodeconnect -n nodeconnect
helm install nodeconnect . --namespace nodeconnect --values values-digitalocean.yaml

# Port-forward for local testing (access at http://localhost:8080)
kubectl port-forward -n nodeconnect svc/nodeconnect-activepieces 8080:80
```

Monitoring: BullMQ Queue Dashboard

Activepieces includes a built-in Bull Dashboard for monitoring job queues.

Enable the Queue UI

Update the queue secrets to enable it:

```
kubectl delete secret nodeconnect-queue-secrets -n nodeconnect

kubectl create secret generic nodeconnect-queue-secrets \
  --namespace nodeconnect \
  --from-literal=AP_QUEUE_MODE=REDIS \
  --from-literal=AP_QUEUE_UI_ENABLED=true \
  --from-literal=AP_QUEUE_UI_USERNAME=admin \
  --from-literal=AP_QUEUE_UI_PASSWORD='YourSecureQueuePassword!'

# Restart pods to pick up the change
kubectl delete pod nodeconnect-activepieces-0 nodeconnect-activepieces-1 -n nodeconnect
```

Access the Dashboard

Open in your browser:

```
https://YOUR_DOMAIN/api/ui
```

The dashboard shows three queues:

Queue	Purpose
workerJobs	Flow execution jobs (triggered flows)
system-job-queue	Internal system tasks (cleanup, piece sync)
runsMetadata	Flow run metadata processing

Click any queue to see Active, Waiting, Completed, Failed, and Delayed jobs with full details.

Connecting to PostgreSQL with DBeaver

Get Connection Details

```
doctl databases connection nodeconnect-postgres
```

Or find them in the DigitalOcean dashboard: **Databases** → **nodeconnect-postgres** → **Connection Details**.

DBeaver Setup

1. Open DBeaver → **New Database Connection** → **PostgreSQL** → **Next**
2. Fill in the **Main** tab:

Field	Value
Host	`your-db-xxxxx.db.ondigitalocean.com`
Port	`25060`
Database	`defaultdb`
Username	`doadmin`
Password	Your DB password (`AVNS_xxxxxxxxxxxxxxx`)

1. Check **Save password**.
2. Go to the **SSL** tab:

- Check **Use SSL**
- Set **SSL mode** to `require`

- Leave CA/Client certificate fields empty
- If you get a "self-signed certificate" error, check **Do not validate certificate**

3. Click **Test Connection** → should show "Connected" → **Finish**

Browse tables under: **defaultdb** → **Schemas** → **public** → **Tables** (you'll see `user`, `project`, `flow`, `flow_run`, etc.)

Monthly Costs (DigitalOcean)

Resource	Cost
DOKS Nodes (2x 2vCPU/4GB)	$48
Managed PostgreSQL	$15
In-cluster Redis	$0 (runs on existing nodes)
Load Balancer	$12
Block Storage (20GB)	$2
Total	**~$77/mo**

All costs are **fixed monthly** regardless of traffic or request volume. You'd only pay more if you:

- Add more nodes (autoscaling)
- Exceed the outbound bandwidth allowance (several TB included, overage at $0.01/GB)
- Upgrade the Postgres plan due to storage growth
- Enable database backups (~$3/mo extra)

Stopping / Tearing Down the Service

Option 1: Scale Down Pods (pause the app, restart in seconds)

Stops the app but keeps everything running. No data loss.

```
# Stop
kubectl scale statefulset nodeconnect-activepieces -n nodeconnect --replicas=0

# Restart
kubectl scale statefulset nodeconnect-activepieces -n nodeconnect --replicas=2
```

Cost: Still ~$77/mo.

Option 2: Delete App + Redis, Keep Cluster + Database

Removes the Helm releases but keeps the cluster and Postgres (data safe).

```
helm uninstall nodeconnect -n nodeconnect
helm uninstall nodeconnect-redis -n nodeconnect
```

Cost: Still ~$75/mo. To redeploy: reinstall Redis (Step 5b), recreate secrets (Step 7), helm install (Step 8).

Option 3: Keep Only Database (saves ~$62/mo)

Preserves your data in Postgres. Everything else is deleted.

```
helm uninstall nodeconnect -n nodeconnect
helm uninstall nodeconnect-redis -n nodeconnect
helm uninstall ingress-nginx -n ingress-nginx
doctl kubernetes cluster delete nodeconnect-cluster --dangerous
```

Cost: **~$15/mo** (managed Postgres only). To redeploy: recreate cluster (Step 4), then Steps 5b–9.

Option 4: Delete Everything ($0/mo)

Completely removes all resources. **Data will be permanently lost.**

```
helm uninstall nodeconnect -n nodeconnect
helm uninstall nodeconnect-redis -n nodeconnect
helm uninstall ingress-nginx -n ingress-nginx
doctl kubernetes cluster delete nodeconnect-cluster --dangerous
doctl databases delete nodeconnect-postgres --force
```

Cost: **$0/mo**. To redeploy: follow the entire guide from Step 4.

Teardown Summary

Option	What's Kept	Monthly Cost	Time to Restart
Scale to 0	Everything	~$77	Seconds
Uninstall app	Cluster + DB	~$75	~10 min
Keep DB only	Postgres only	~$15	~20 min
Delete all	Nothing	$0	~30 min

Next Steps

1. **Database backups** — Enable automatic backups for PostgreSQL in the DO dashboard (~$3/mo extra).
2. **Proper SSL for Postgres** — Download the DO CA certificate and set `AP_POSTGRES_SSL_CA` instead of `NODE_TLS_REJECT_UNAUTHORIZED=0`.
3. **Monitoring** — Add DigitalOcean Monitoring or Prometheus/Grafana.
4. **Scaling** — Enable `autoscaling` in `values-digitalocean.yaml` for auto-scaling based on CPU.
5. **CI/CD** — Set up GitHub Actions to auto-build and deploy on push to main.